

Predicting Book Ratings: An End-to-End Machine Learning Pipeline

Nahasat Nibir
DSTI School of Engineering / Data Wrangling with SQL

June 2026

1 Introduction

The goal of this project is to build a machine learning model that predicts the average rating of a book (on a 0–5 scale) based on its metadata, such as title, authors, page count, publisher, language, and engagement metrics (ratings count, reviews count). The dataset contains 11,128 books with 12 attributes. After cleaning and feature engineering, we trained and compared multiple regression models, ultimately deploying the best one as a Streamlit web application.

This report details the full pipeline: exploratory data analysis, data cleaning, feature engineering, model selection and evaluation, and deployment.

2 Methodology

2.1 Exploratory Data Analysis

The raw dataset (11,128 rows) was examined for distributions, missing values, and anomalies. - The target variable `average_rating` exhibited a left-skewed distribution with a mean of approximately 4.0. - `num_pages`, `ratings_count`, and `text_reviews_count` all showed heavy right-skewness, requiring log-transformation. - Several date formats were present in `publication_date`, and language codes included variants (e.g., `en-US`, `eng`). - The `authors` field contained multiple authors separated by “/”.

Figure 1 shows the correlation matrix of numeric features before cleaning. Outliers were detected via the IQR method and capped at the 99th percentile.

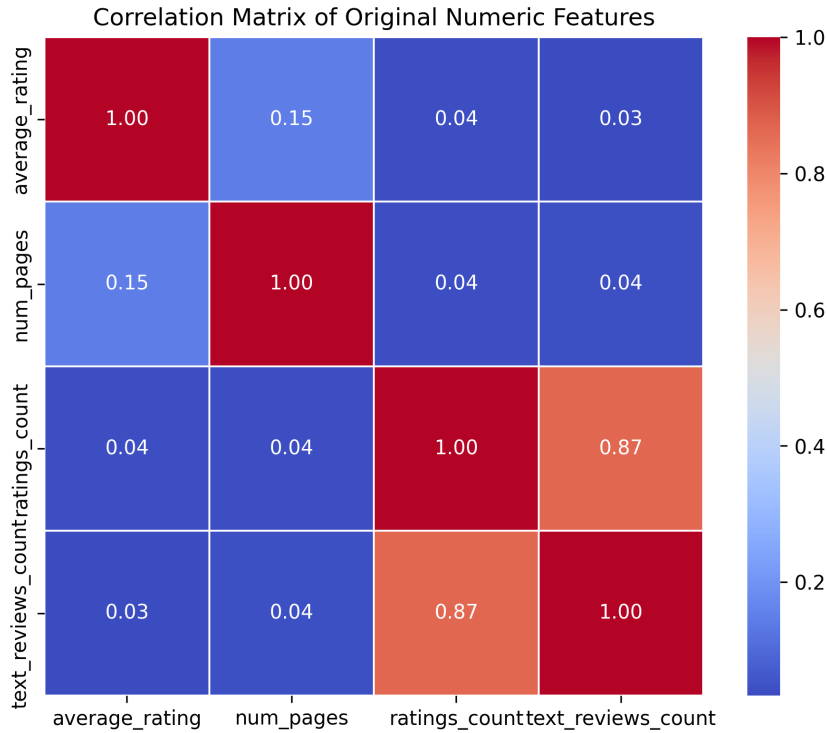


Figure 1: Correlation matrix of original numeric features.

2.2 Data Cleaning

The following cleaning steps were applied:

- Parsed `publication_date` with `pd.to_datetime` (coercing errors); 2 rows with unparseable dates were dropped.
- Standardised language codes to ISO 639-2 (e.g., `en-US` → `eng`).
- Split `authors` on “/” and created `author_count` and `primary_author` features.
- Capped `num_pages` at the 99th percentile (1174.7) to reduce the influence of extreme outliers.
- Dropped rows with missing `average_rating`.

After cleaning, the dataset contained 11,117 books.

2.3 Feature Engineering

From the cleaned data, we engineered the following features:

- `title_length`, `title_word_count`, `has_exclamation` (from `title`).
- `publication_year`, `book_age` (relative to 2026), `is_classic` (`age > 50`).
- `log_ratings_count`, `log_text_reviews_count`, `reviews_per_rating`.
- Smoothed target encoding for high-cardinality categoricals: `publisher_te`, `primary_author_te`, `language_code_te`.

Identifiers (`bookID`, `isbn`, `isbn13`) and raw text (`title`) were dropped.

2.4 Feature Selection

Mutual information regression and Recursive Feature Elimination (RFE) with a Ridge estimator were used to select the most predictive features. The final set of 8 features was: `num_pages`, `ratings_count`, `text_reviews_count`, `log_ratings_count`, `log_text_reviews_count`, `reviews_per_rating`, `publisher_te`, `primary_author_te`.

2.5 Model Selection and Comparison

We trained and compared:

- Baselines: Dummy Regressor, Linear Regression, Ridge, Lasso.
- Advanced models: Random Forest, XGBoost, LightGBM, CatBoost.

All models were evaluated using 5-fold cross-validation (RMSE) and a hold-out test set (80/20 split, random state 42). Hyperparameter tuning was performed with `RandomizedSearchCV` (20 iterations per model).

3 Results

3.1 Model Performance

Table 1 summarises the performance of all models.

Table 1: Model performance comparison (test set and CV).

Model	CV RMSE	Test RMSE	Test R^2
Dummy Regressor	0.3502	0.3253	0.0000
Linear Regression	0.3034	0.2888	0.2114
Ridge	0.3034	0.2887	0.2119
Lasso	0.3502	0.3253	0.0000
Random Forest	0.2864	0.2857	0.2281
LightGBM	0.2877	0.2848	0.2330
CatBoost	0.2904	0.2813	0.2517
XGBoost	0.2905	0.2853	0.2303
Ensemble (RF+LGBM)	–	0.2841	0.2369

The simple average ensemble of Random Forest and LightGBM achieved the lowest test RMSE (0.2841) and was selected as the final model.

3.2 Residual Analysis

Figure 2 shows the residual plot for the ensemble model. The errors are roughly symmetrically distributed around zero, with a slight tendency to underpredict very high ratings (> 4.5) and overpredict low ratings (< 3.0), which is expected given the limited representation of extreme ratings in the dataset.

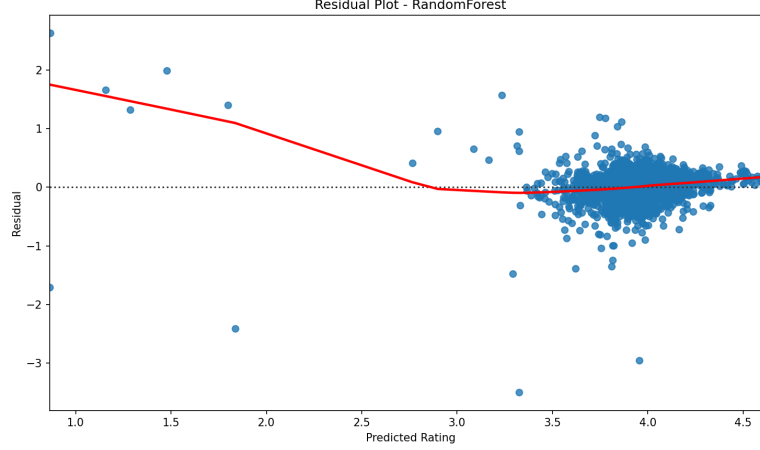


Figure 2: Residual plot for the Random Forest model (similar to ensemble).

3.3 Feature Importance

Figure 3 displays the feature importances from the Random Forest model. The target-encoded author feature (`primary_author_te`) is by far the strongest predictor, followed by the raw engagement counts. This indicates that the author’s historical average rating and the popularity of the book strongly influence its rating – a plausible pattern, though `ratings_count` and `text_reviews_count` are post-publication features and constitute a form of target leakage if we were to predict ratings before the book accumulates reviews. This is discussed further in the Conclusion.

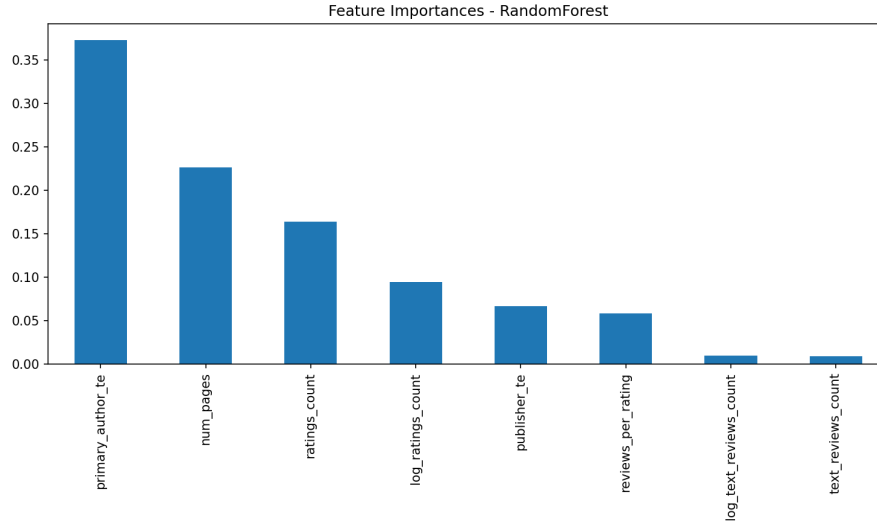


Figure 3: Feature importances from Random Forest.

3.4 Interpretability with SHAP

SHAP summary plots (not shown due to space) confirmed that `primary_author_te` and `publisher_te` have the largest impact on predictions, with higher author/publisher mean ratings pushing the prediction upward. `log_ratings_count` and `num_pages` also show positive effects, while `reviews_per_rating` has a mixed influence.

3.5 Error by Rating Band

Figure 4 shows that the model performs best on books rated 4–5 (RMSE ≈ 0.22) and worst on books rated below 3 (RMSE ≈ 0.45). This is partly due to class imbalance: the majority of books in the dataset are highly rated.

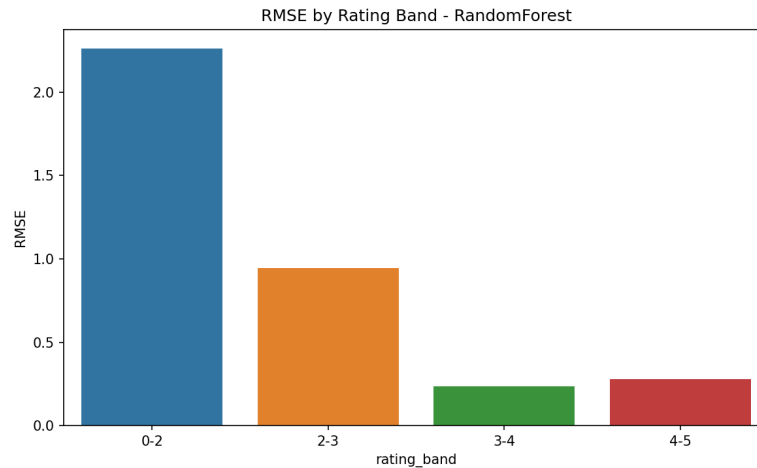


Figure 4: RMSE by rating band.

4 Deployment

A Streamlit web application was built to serve the final model. The user interface offers two modes:

- **Single Book:** a form with all required fields (title, authors, pages, ratings count, etc.). After submission, the predicted rating is displayed on a styled card.
- **Batch CSV Upload:** users upload a CSV file with the same columns as the original dataset. The app validates the schema, runs predictions on all rows, and provides a downloadable CSV with an additional `predicted_rating` column.

The app loads the full preprocessing + model pipeline from a single serialised file (`best_pipeline.joblib`), ensuring exact reproduction of the training-time transformations. The architecture is summarised in Figure 5.

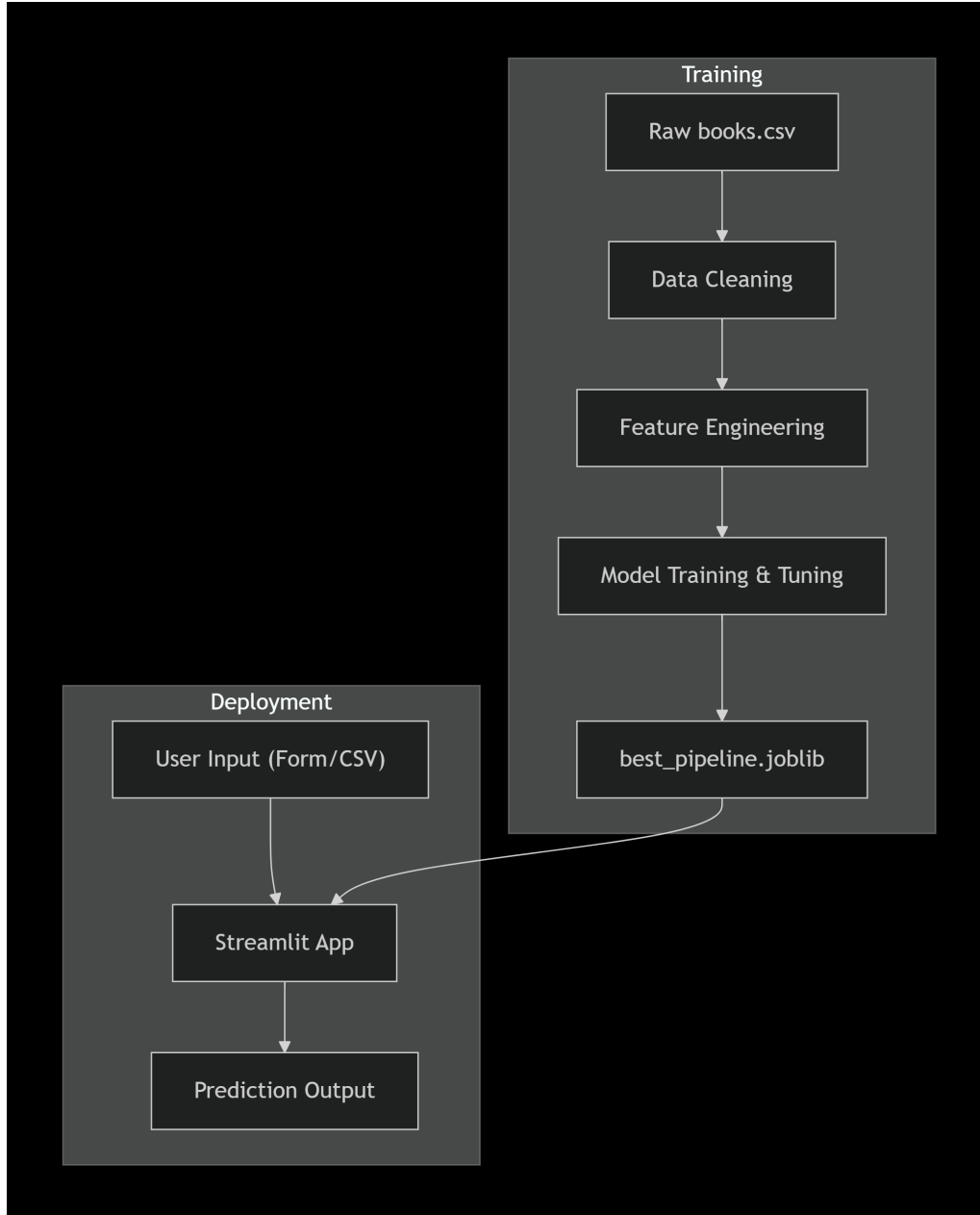


Figure 5: System architecture: training in Colab, deployment on Windows with Streamlit.

5 Conclusion

We successfully built a machine learning pipeline that predicts book ratings with a test RMSE of 0.284 and an R^2 of 0.237. The ensemble of Random Forest and LightGBM provided the best performance, and the model was deployed as an interactive Streamlit application.

5.1 Limitations and Future Work

- **Target leakage:** `ratings_count` and `text_reviews_count` are post-publication features. For a true cold-start prediction (before a book gains reviews), these would not be available. A future version could omit these features and rely solely on author/publisher reputation and textual features from the title.
- **Text features:** Only basic title-derived features were used. Employing TF-IDF or

pre-trained embeddings on the title (and potentially the book description, if available) could capture genre and style information.

- **Data size:** With only 11K books, more complex models like neural networks or transformer-based embeddings would likely overfit. Gathering a larger dataset or using data augmentation could improve performance.
- **Hyperparameter tuning:** More exhaustive search (more iterations, wider grids) may yield slight improvements, but the current gains are marginal given the dataset size.

The full code, trained model, and instructions are available on GitHub: https://github.com/Nibir1/Book_Rating_Prediction_ML_Pipeline.